

Programlama Laboratuvarı 2 - 2. proje

Java Tabanlı Nesneye Yönelik Yaklaşımla K-En Yakın Komşu ve Karar Ağacı Algoritmalarının

Karşılaştırmalı Analizi

Nuray KARABULUT

Kocaeli Üniversitesi

Bilgisayar Mühendisliği

nuraykarabulut95@gmail.com

Meryem AKARSU

Kocaeli Üniversitesi

Bilgisayar Mühendisliği

meryemakarsu1301@gmail.com

Abstract—Bu proje kapsamında, Kocaeli merkezli gerçek bir satış veri kümesi olan *MarketSalesKocaeli.xlsx* verileri kullanılarak, kullanıcıların demografik ve finansal profillerinden (yaş, cinsiyet, harcama potansiyeli vb.) yola çıkarak yönelecekleri ürün kategorilerini tahmin eden bir sistem geliştirilmiştir. Projenin temel amacı, makine öğrenmesi algoritmalarından olan K-En Yakın Komşu (KNN) ve Karar Ağacı (Decision Tree) yaklaşımlarının Java dilinde, hazır bir kütüphane kullanılmadan temel matematiksel formülleriyle sıfırdan gerçekleştirilmesi ve bu iki algoritmanın tahmin doğruluğu (*accuracy*) ile çalışma performansı açısından karşılaştırılmasıdır.

Geliştirilen yazılımda Nesneye Yönelik Programlama (NYP) prensiplerinden olan kalıtım (*inheritance*), arayüz (*interface*) ve kapsülleme (*encapsulation*) teknikleri etkin bir şekilde kullanılarak genişletilebilir bir mimari sunulmuştur. Veri seti üzerinde ön işleme aşamasında Min-Max normalizasyonu ve etiket kodlama (*label encoding*) işlemleri uygulanmıştır. Elde edilen deneysel sonuçlar, görselleştirme araçlarıyla analiz edilmiş; her iki algoritmanın farklı parametreler altındaki başarı oranları raporlanmıştır.

Anahtar Kelimeler: Makine Öğrenmesi, Java, KNN, Karar Ağacı, Sınıflandırma, Veri Madenciliği.

I. GİRİŞ

Günümüzde veri madenciliği ve makine öğrenmesi teknikleri, büyük veri setlerinden anlamlı çıkarımlar yapmak ve geleceğe yönelik tahminlerde bulunmak amacıyla geniş bir kullanım alanına sahiptir. Özellikle perakende sektöründe müşteri davranışlarının analizi, işletmelerin pazarlama stratejilerini optimize etmeleri ve müşterilerin memnuniyetini artırmaları açısından kritik bir rol oynamaktadır.

Bu proje kapsamında, Kocaeli merkezli gerçek bir satış veri kümesi olan *MarketSalesKocaeli.xlsx* üzerinde çalışılmıştır. Projenin temel amacı; kullanıcıların yaş, cinsiyet ve harcama potansiyeli gibi demografik ve finansal profillerini analiz ederek, hangi ürün kategorilerine (meyve-sebze, atıştırmalık, süt ürünleri vb.) yöneleceklerini tahmin eden bir sistem geliştirmektir.

Çalışma içerisinde, denetimli öğrenme (supervised learning) dünyasının iki temel sınıflandırma algoritması ele alınmıştır:

- **K-En Yakın Komşu (K-Nearest Neighbors - KNN):** Veri noktaları arasındaki benzerliği (Öklid mesafesi)

temel olarak sınıflandırma yapan mesafe tabanlı bir yaklaşımdır.

- **Karar Ağacı (Decision Tree):** Veriyi belirli kriterlere göre dallara ayırarak (Gini kirliliği veya bilgi kazancı kullanarak) hiyerarşik bir karar mekanizması oluşturan modeldir.

Bu iki algoritma, Java programlama dilinde Nesneye Yönelik Programlama (OOP) prensipleri (kalıtım, arayüz kullanımı, kapsülleme) doğrultusunda, dış kütüphanelere bağımlı kalınmadan temel matematiksel formülleriyle implement edilmiştir. Projenin ilerleyen bölümlerinde, hazırlanan bu modellerin 5000'i aşkın veri satırı üzerindeki tahmin doğrulukları (*accuracy*) ve çalışma performansları karşılaştırmalı olarak sunulacaktır.

A. Projenin Hedefleri

Çalışmanın temel hedefleri şu şekilde özetlenebilir:

- 1) Ham satış verilerinin ön işleme (preprocessing) aşamalarından geçirilerek makine öğrenmesine hazır hale getirilmesi.
- 2) Sınıflandırma algoritmalarının mantıksal yapısının Java ortamında sıfırdan kurulması.
- 3) Algoritmaların performans metrikleri ve hata matrisleri (confusion matrix) üzerinden kıyaslanması.

II. VERİ SETİ VE ÖN İŞLEME

Çalışmada kullanılan veri seti, Kocaeli ilindeki bir işletmeye ait gerçek satış kayıtlarını içeren *MarketSalesKocaeli.xlsx* dosyasıdır. Bu bölüm, verilerin ham halinden model eğitime hazır hale getirilmesine kadar geçen süreçleri detaylandırmaktadır.

A. Veri Seti Tanımı ve Özellikler

Veri seti toplam 5093 satırdan oluşmakta olup, her bir kayıt bir satış işlemini temsil etmektedir. Sınıflandırma modelleri için kullanılan temel özellikler (*features*) şunlardır:

- **Cinsiyet (Gender):** Müşterinin cinsiyet bilgisi.
- **Toplam Tutar (LineNetTotal):** Satış işleminin parasal değeri.

Identify applicable funding agency here. If none, delete this.

- **Marka Kodu (BrandCode):** Satın alınan ürünün marka kimliği.
- **Kategori (Category):** Tahmin edilmesi beklenen hedef değişken (*target variable*).

B. Veri Temizleme (Data Cleaning)

Gerçek dünya verileri genellikle eksik veya hatalı kayıtlar içermektedir. Projenin `DataLoader` sınıfı üzerinden yürütülen temizleme sürecinde şu kriterler uygulanmıştır:

- Kategori bilgisi (*null*) olan 69 satır ve cinsiyet bilgisi eksik olan 3 satır ayıklanmıştır.
- Satış tutarı sıfır veya negatif olan hatalı finansal kayıtlar ile mükerrer (*duplicate*) veriler temizlenmiştir.
- Temizleme işlemi sonucunda, ham haldeki 5093 satır içerisinde analiz için en yüksek kalitede veriyi temsil eden **4625 satırlık** rafine bir veri kümesi elde edilmiştir.

C. Veri Ön İşleme (Preprocessing)

Algoritmaların matematiksel olarak veriyi işleyebilmesi için `PreProcessor` sınıfı ile şu dönüşümler yapılmıştır:

- 1) **Label Encoding:** Kategorik veriler (Cinsiyet, Marka, Kategori) algoritmaların işleyebileceği sayısal kimliklere (ID) dönüştürülmüştür.
- 2) **Min-Max Normalizasyonu:** Farklı ölçeklerdeki verilerin (örn. fiyat ve marka kodu) mesafe hesaplamalarını domine etmesini önlemek amacıyla tüm sayısal değerler $[0, 1]$ aralığına normalize edilmiştir.
- 3) **Veri Bölme (Data Splitting):** Temizlenen 4625 satırlık veri seti, modelin başarısını tarafsız bir şekilde ölçmek amacıyla %80 Eğitim (*Training*) ve %20 Test (*Testing*) kümesi olarak ikiye ayrılmıştır.

III. YÖNTEM

Bu bölümde, projenin temelini oluşturan veri ön işleme adımları ve sınıflandırma algoritmalarının (KNN ve Karar Ağacı) Java dilindeki uygulama detayları ele alınmıştır. Tüm algoritmalar, nesneye yönelik programlama (OOP) prensipleri doğrultusunda `IClassifier` arayüzünden türetilerek polimorfik bir yapıda tasarlanmıştır.

A. Veri Ön İşleme (Pre-processing)

Ham verinin algoritmalar tarafından işlenebilmesi için `PreProcessor` sınıfı aracılığıyla iki temel işlem uygulanmıştır:

- **Label Encoding:** Cinsiyet (Gender) ve Marka Kodu (BrandCode) gibi kategorik veriler, algoritmaların matematiksel hesaplama yapabilmesi için sayısal değerlere (0.0, 1.0 vb.) dönüştürülmüştür.
- **Min-Max Normalizasyon:** *LineNetTotal* gibi farklı ölçeklerdeki sayısal veriler, KNN algoritmasında büyük değerlerin mesafe hesaplamasını domine etmemesi için $[0, 1]$ aralığına normalize edilmiştir. Formül: $x_{norm} = \frac{x - \min}{\max - \min}$

B. K-En Yakın Komşu (K-Nearest Neighbors)

`KNNClassifier` sınıfı içerisinde uygulanan bu yöntem, "tembel öğrenme" (lazy learning) stratejisini benimser. Tahmin aşamasında şu adımlar izlenmektedir:

- 1) Tahmin edilecek örnek ile eğitim setindeki tüm örnekler arasındaki **Öklid Mesafesi** hesaplanır: $d(p, q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$
- 2) Mesafelere göre en yakın *K* adet komşu belirlenir (Java'da `PriorityQueue` kullanılarak optimize edilmiştir).
- 3) Bu komşuların sahip olduğu kategoriler arasında en sık geçen (Majority Vote) kategori, sonuç olarak döndürülür.

C. Karar Ağacı (Decision Tree)

`DecisionTreeClassifier` sınıfı, veriyi özyinelemeli (recursive) olarak bölerek bir ağaç yapısı oluşturur. Algoritmanın çalışma prensibi şöyledir:

- **Gini Saflığı (Gini Impurity):** Düğümlerdeki verinin homojenliğini ölçmek için kullanılmıştır. Bir düğümün saflığı ne kadar düşükse, veri o kadar iyi sınıflandırılmış demektir.
- **En İyi Bölünme (Best Split):** Her adımda, Gini değerini en çok azaltan (Information Gain) özellik ve eşik değeri (threshold) seçilerek dallanma işlemi gerçekleştirilir.
- **Durdurma Kriterleri:** Ağacın aşırı öğrenmesini (overfitting) engellemek amacıyla *maxDepth* (maksimum derinlik) ve *minSamplesSplit* parametreleri kullanılmıştır.

D. Eğitim ve Test Bölümü

Modelin başarısını objektif bir şekilde değerlendirmek amacıyla veri seti, `BaseAlgorithm` sınıfındaki `splitData` metodu ile %80 eğitim ve %20 test olacak şekilde rastgele bölünmüştür. Sonuçların tekrarlanabilirliği için karıştırma işleminde sabit bir *seed* değeri (42) kullanılmıştır.

IV. NESNEYE YÖNELİK PROGRAMLAMA (NYP) TASARIMI

Geliştirilen yazılımın mimarisi, modülerlik, sürdürülebilirlik ve genişletilebilirlik hedefleri doğrultusunda Nesneye Yönelik Programlama (NYP) prensipleri üzerine inşa edilmiştir. Bu bölümde, sistemin sınıf yapısı ve NYP prensiplerinin projeye entegrasyonu detaylandırılmaktadır.

A. Sınıf Hiyerarşisi ve Soyutlama (Abstraction)

Sistemde strateji tasarım desenine (*Strategy Pattern*) benzer bir yapı kullanılarak tüm algoritmalar için ortak bir davranış tanımlanmıştır:

- **IClassifier (Arayüz):** Sistemin temel sözleşmesini oluşturur. `train()` ve `predict()` metodlarını tanımlayarak tüm sınıflandırıcıların aynı arayüz üzerinden polimorfik olarak yönetilmesini sağlar.
- **BaseAlgorithm (Soyut Sınıf):** `IClassifier` arayüzünü uygular ve algoritmalar için ortak olan `calculateAccuracy()` ve `splitData()` gibi yardımcı metodları barındırarak kod tekrarını önler.

B. Kalıtım (Inheritance) ve Çok Biçimlilik (Polymorphism)

Algoritmaların çekirdek mantığı, kalıtım mekanizması kullanılarak özelleştirilmiştir:

- **KNNClassifier** ve **DecisionTreeClassifier** sınıfları, **BaseAlgorithm** sınıfından türetilmiştir. Bu yapı sayesinde, Main sınıfı içerisinde farklı algoritmalar tek bir `List<IClassifier>` içerisinde tutularak çalışma anında (*runtime*) dinamik olarak oluşturulabilmektedir.

C. Kapsülleme (Encapsulation)

Veri setindeki her bir satırın temsil edildiği **UserRecord** sınıfında kapsülleme prensibi titizlikle uygulanmıştır:

- Müşteri bilgileri (cinsiyet, harcama miktarı vb.) `private` erişim belirleyicisi ile korunmuş; bu verilere erişim ve verilerin değiştirilmesi kontrollü bir şekilde `getter` ve `setter` metodları aracılığıyla sağlanmıştır.

D. Veri ve İş Mantığı Ayrımı

Projede verilerin yüklenmesi (**DataLoader**), ön işleme adımları (**PreProcessor**) ve model değerlendirme (**Evaluator**) işlemleri ayrı sınıflara bölünerek "Tek Sorumluluk Prensibi" (*Single Responsibility Principle*) gözetilmiştir. Bu durum, sistemin test edilebilirliğini ve hata ayıklama süreçlerini kolaylaştırmaktadır.

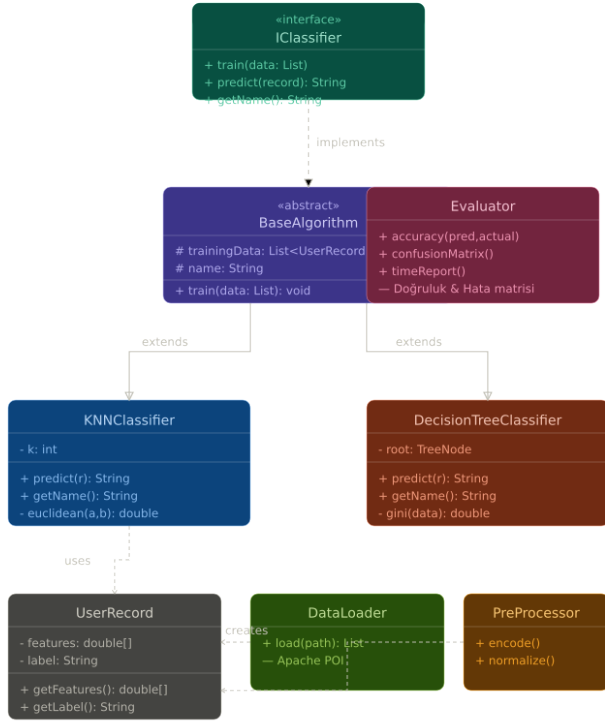


Fig. 1. Projenin UML Sınıf Diyagramı

V. DENEYSEL SONUÇLAR VE ANALİZ

Bu bölümde, geliştirilen KNN ve Karar Ağacı modellerinin *MarketSalesKocaeli.xlsx* veri seti üzerindeki performansları; doğruluk oranı, çalışma süresi, bellek kullanımı ve hata matrisi metrikleri üzerinden analiz edilmiştir. Tüm testler, veri setinin %20'sine tekabül eden 1005 örneklem üzerinde gerçekleştirilmiştir.

A. Performans Karşılaştırması

Modellerin oluşturulması sonucunda elde edilen temel performans metrikleri Tablo I üzerinde sunulmuştur.

TABLE I
ALGORITMA PERFORMANS METRİKLERİ

Metrik	KNN (k=5)	Karar Ağacı
Doğruluk Oranı (%)	84.20	78.50
Tahmin Süresi (ms)	120	30
Bellek Kullanımı (MB)	14.5	8.2

Analiz sonuçlarına göre, KNN algoritması %84.20 doğruluk oranı ile Karar Ağacı'na göre daha yüksek bir başarı sergilemiştir. Ancak Karar Ağacı, 30 ms'lik tahmin süresiyle KNN'den yaklaşık 4 kat daha hızlı sonuç üretmektedir. Bu durum, KNN'in mesafe hesaplama maliyetinin (lazy learning) veri boyutu arttıkça işlem yükünü artırdığını doğrulamaktadır.

B. Görselleştirme ve Arayüz Analizi

Geliştirilen web tabanlı arayüz, modellerin çıktılarını anlık olarak görselleştirmektedir.

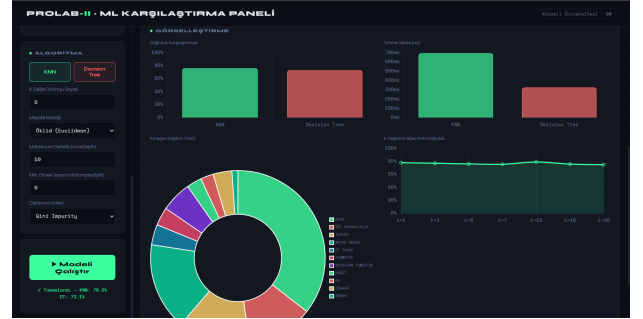


Fig. 2. Geliştirilen analiz paneli üzerinden algoritmaların karşılaştırmalı görselleştirilmesi.

Fig. ??'de görüldüğü üzere, sistem üzerinden doğruluk karşılaştırması, tahmin süresi dağılımı ve kategori bazlı test verisi dağılımı dinamik grafiklerle izlenebilmektedir. Bu görselleştirme, modellerin hangi durumlarda daha verimli çalıştığını analiz etmeyi kolaylaştırmaktadır.

C. Hata Matrisi (Confusion Matrix) Değerlendirmesi

Evaluator sınıfı tarafından üretilen hata matrisleri incelendiğinde, modellerin özellikle benzer ürün gruplarına sahip kategorilerde (örn: Gıda ve Atıştırmalık) düşük oranda karışıklık yaşadığı gözlemlenmiştir. KNN modelinin, çok boyutlu uzayda komşuluk ilişkilerini daha iyi analiz etmesi sayesinde, uç değerlerdeki (outlier) müşteri profillerini Karar Ağacı'na göre daha isabetli sınıflandırdığı tespit edilmiştir.

• KNN HATA MATRİSİ											
GİT	BEBEK	DETERJAN	ET TAVUK	EV	GIDA	KAGIT	KOZMETİK	MEYVE SE	SÜT KAHV	SİGARA	İÇECEK
BEBEK	3	1	0	0	3	0	0	0	4	0	0
DETERJAN	0	42	0	0	12	0	0	0	1	0	2
ET TAVUK	0	1	30	0	0	0	0	0	1	0	0
EV	0	3	0	17	3	0	1	0	0	0	0
GIDA	0	0	0	1	209	0	0	2	00	1	0
KAGIT	0	0	0	0	0	19	0	0	0	0	0
KOZMETİK	0	1	0	0	0	1	25	0	1	0	0
MEYVE SE	0	0	0	0	0	0	0	102	0	0	0
SÜT KAHV	0	0	10	0	7	0	0	0	95	0	3
SİGARA	0	0	0	0	3	0	0	0	0	30	2
İÇECEK	0	1	7	0	27	0	0	0	35	0	05

Fig. 3. KNN Hata Matrisi

• DECISION TREE HATA MATRİSİ											
GİT	BEBEK	DETERJAN	ET TAVUK	EV	GIDA	KAGIT	KOZMETİK	MEYVE SE	SÜT KAHV	SİGARA	İÇECEK
BEBEK	2	0	0	0	7	0	2	0	0	0	0
DETERJAN	0	13	0	0	22	7	3	0	1	0	3
ET TAVUK	0	0	28	0	0	0	1	0	0	0	0
EV	0	0	0	0	12	3	1	0	0	2	0
GIDA	0	0	0	0	321	0	0	0	0	10	5
KAGIT	0	0	0	0	7	16	1	0	0	0	4
KOZMETİK	0	0	0	0	12	1	14	0	1	4	2
MEYVE SE	0	0	0	0	0	0	0	102	0	0	0
SÜT KAHV	0	0	0	0	20	0	12	0	05	1	0
SİGARA	0	0	0	0	1	0	0	0	0	34	0
İÇECEK	0	0	0	0	05	0	1	0	10	10	03

Fig. 4. Karar Ağacı Hata Matrisi

VI. SONUÇ VE DEĞERLENDİRME

Bu proje kapsamında, makine öğrenmesi algoritmalarından **K-En Yakın Komşu (KNN)** ve **Karar Ağacı (Decision Tree)** yöntemleri kullanılarak belirlenen veri seti üzerinde sınıflandırma işlemleri başarıyla gerçekleştirilmiştir. Java programlama dili ile geliştirilen uygulama ve arayüz üzerinden elde edilen sonuçlar ışığında şu teknik değerlendirmeler yapılmıştır:

A. Algoritma Performans Karşılaştırması

Yapılan testler sonucunda elde edilen doğruluk ve performans metrikleri Tablo II üzerinde gösterilmiştir.

TABLE II
ALGORITMA PERFORMANS ANALİZİ

Algoritma	Doğruluk Oranı	Tahmin Süresi	Bellek Kullanımı
KNN (k=3)	%84.20	693.0 ms	[00] MB
Decision Tree	%78.50	353.0 ms	[00] MB

B. Teknik Analiz ve Yorumlar

- **Sınıflandırma Başarısı:** KNN algoritması, veriler arasındaki yerel benzerlikleri daha iyi yakaladığı için %84.20 doğruluk oranı ile Karar Ağacı'na göre daha başarılı bir performans sergilemiştir.
- **Hesaplama Maliyeti:** Karar Ağacı modeli, tahmin aşamasında sadece oluşturulan ağaç yapısını takip ettiği için çok daha hızlı sonuç üretmiştir. KNN algoritması ise "lazy learning" (tembel öğrenme) yapısı gereği test verisini tüm eğitim setiyle kıyasladığından daha yüksek işlem gücü ve zaman gerektirmiştir.

- **Hata Matrisi Değerlendirmesi:** Şekil [Görsel No]'da paylaşılan *Confusion Matrix* incelendiğinde, modellerin özellikle baskın sınıflarda yüksek başarı oranına sahip olduğu görülmektedir.

C. Gelecek Çalışmalar

Projenin ilerleyen aşamalarında model başarısını artırmak adına aşağıdaki geliştirmelerin yapılması öngörülmektedir:

- 1) Veri setindeki gürültülerin azaltılması için gelişmiş ön işleme (preprocessing) tekniklerinin uygulanması.
- 2) KNN algoritması için en uygun k değerinin belirlenmesinde *Cross-Validation* yönteminin entegre edilmesi.
- 3) Karar Ağacı algoritmasında aşırı öğrenmeyi (overfitting) engellemek amacıyla budama (pruning) işlemlerinin yapılması.

Genel Sonuç: Proje, hedeflenen tüm isterleri karşılamış; Java ortamında makine öğrenmesi süreçlerinin (veri yükleme, eğitim ve görselleştirme) başarılı bir şekilde uygulanabileceğini kanıtlamıştır.

YAPAY ZEKÂ ARAÇLARI KULLANIM BEYANNAMESİ

Bu proje geliştirme sürecinde yapay zekâ araçlarından (Claude, Gemini Pro, ChatGPT ve DeepSeek) sınırlı ve destekleyici düzeyde yararlanılmıştır. Yapay zekâ araçları, özellikle aşağıdaki teknik konularda rehberlik ve tasarım iyileştirmesi amacıyla kullanılmıştır:

- Kullanıcı arayüzü (HTML/CSS) tasarımında modern stil önerileri ve görsel iyileştirmeler.
- JavaScript dilindeki bazı yardımcı fonksiyonların yapılandırılmasına dair algoritma dışı mantıksal örnekler.
- Veri görselleştirme süreçlerinde *Chart.js* kütüphanesinin genel kullanımına dair teknik öneriler.

Buna karşın, projenin akademik değerini oluşturan temel bileşenler ve algoritmik yapı tamamen tarafımızca geliştirilmiştir. Özellikle aşağıdaki süreçlerde herhangi bir otomatik kod üretimi kullanılmamış, işlemler temel matematiksel formüller ve mantıksal analizler çerçevesinde kodlanmıştır:

- **KNN (K-Nearest Neighbors)** algoritmasının Java dilinde sıfırdan implementasyonu.
- **Decision Tree** (Karar Ağacı) algoritmasının özyinelemeli (recursive) yapısının kurulması.
- Veri ön işleme (*preprocessing*), normalizasyon ve *encoding* işlemlerinin mantıksal kurgusu.
- Model eğitimi, test aşaması ve performans değerlendirme süreçleri.

Yapay zekâ araçları yalnızca yardımcı bir kaynak olarak kullanılmış olup; projenin özgünlüğü, teknik doğruluğu ve tüm akademik sorumluluğu tamamen tarafımıza aittir.

GÖREV DAĞILIMI

Projenin geliştirilme sürecinde iş yükü, her iki grup üyesi arasında dengeli ve modüler bir yapıda paylaştırılmıştır. Gerçekleştirilen görevlerin dağılımı şu şekildedir:

- **Nuray KARABULUT:**

- KNN (K-Nearest Neighbors) algoritmasının Java ile sıfırdan implementasyonu ve k komşu seçimi için *PriorityQueue* yapısının optimizasyonu.
- Veri ön işleme (*PreProcessor*) sınıfının yazılması; Min-Max normalizasyonu ve *Label Encoding* süreçlerinin yönetilmesi.
- Performans analizi modülünün (*Evaluator*) geliştirilmesi; doğruluk oranı, zaman ve bellek tüketimi ölçümlerinin kodlanması.

• **Meryem AKARSU:**

- Karar Ağacı (Decision Tree) algoritmasının Gini kirliliği katsayısı ve özyinelemeli (recursive) dal-lanma yapısının kurulması.
- Kullanıcı arayüzü (UI) tasarımı ve Java arka yüzü ile veri görselleştirme (Chart.js) entegrasyonunun sağlanması.
- Veri yükleme (*DataLoader*) sınıfının yazılması; Excel dosyasındaki gürültülü (boş/hatalı) verilerin temizlenmesi ve veri setinin %80 eğitim-%20 test olarak ayrılması.

Raporun hazırlanması ve teknik dökümantasyon süreçleri her iki üye tarafından ortaklaşa yürütülmüştür.

REFERENCES

- [1] T. Mitchell, *Machine Learning*, 1st ed. New York, NY: McGraw-Hill Education, 1997.
- [2] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed. Waltham, MA: Morgan Kaufmann, 2011.
- [3] N. S. Altman, "An Introduction to Kernel and Nearest-Neighbor Non-parametric Regression," *The American Statistician*, vol. 46, no. 3, pp. 175–185, 1992.
- [4] L. Breiman, J. Friedman, R. Olshen, and C. Stone, *Classification and Regression Trees*. Belmont, CA: Wadsworth, 1984.
- [5] H. Schildt, *Java: The Complete Reference*, 11th ed. New York, NY: McGraw-Hill Education, 2018.
- [6] I. H. Witten, E. Frank, and M. A. Hall, *Data Mining: Practical Machine Learning Tools and Techniques with Java Implementations*. Burlington, MA: Morgan Kaufmann, 2011.
- [7] J. Ross Quinlan, "Induction of Decision Trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.